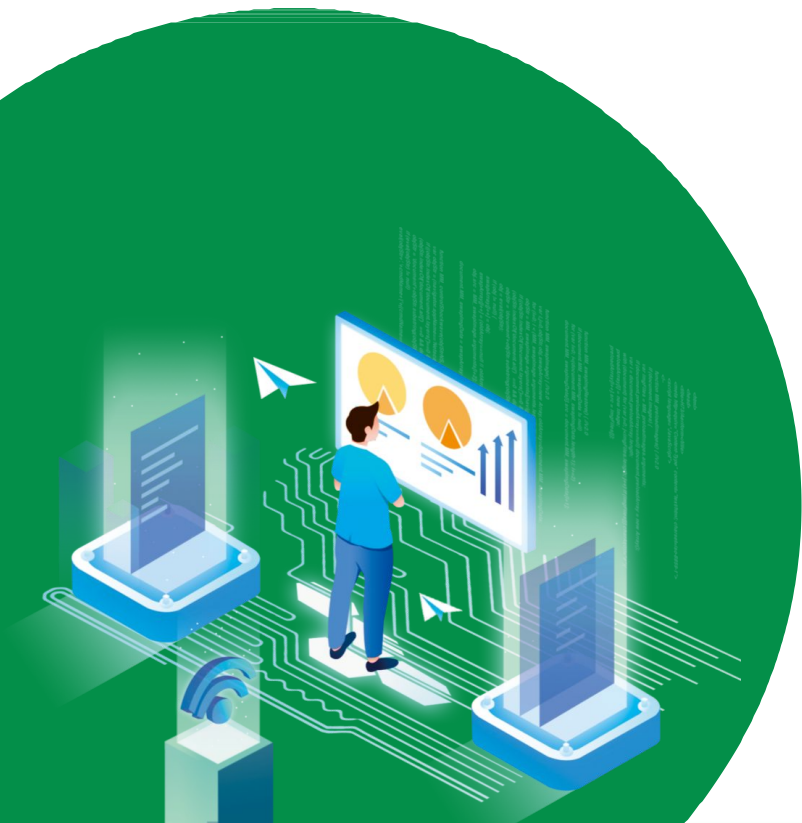


Postgres Optimization (Dev-mode)

Presenter: Link



CONTENT

- 01.** Database: Under the hood
- 02.** Query optimization 101
- 03.** Use cases

BEFORE START



Optimize Database Level

- Config (shared_buffers, effective_cache_size, work_mem, max_wal_size, max_parallel_workers, maintenance_work_mem...)
- Storage engine
- Table structure: Index, Partition, Data Type, Schema Design,...

Optimize OS/Hardware Level

- Disk (HDD, SSD, NVMe,...)
- CPU, Mem,...
- OS, Page size, IOPS, QPS, RAID,...

**DEV ARE
HERE!**

Optimize Application Level

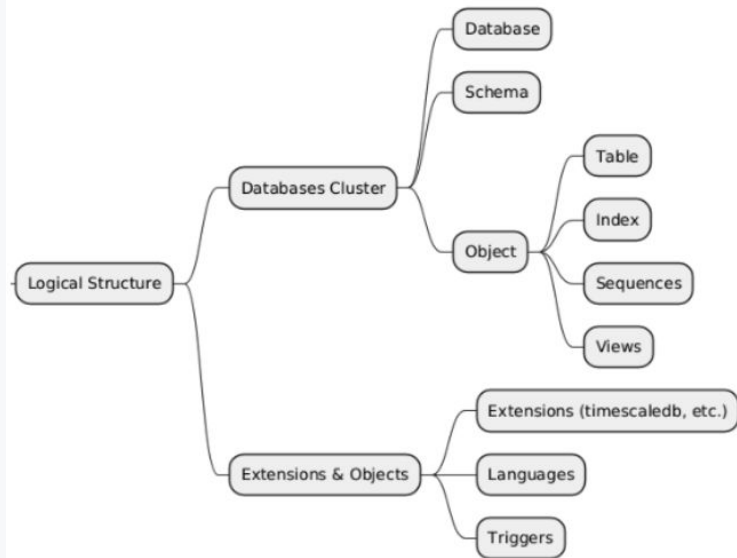
- *Query Optimization (SELECT, INSERT, UPDATE, DELETE, TRUNCATE,...)*
- Transaction, Locking, ORM, Connection Pool,...



The relations between database objects and their respective OIDs are stored in **system catalogs**, depending on the type of objects. For example, OIDs of databases and heap tables are stored in **pg_database** and **pg_class**

```
✓ SELECT datname, oid FROM pg_database;
```

```
rw@noodles-staging 2 of 3
> external 1 of 13
  internal 4 of 16
    coin_indexer
    dex_indexer
      tables 12
      views 17
    noodles_oltp
    public
  Database Objects
    access methods 7
    casts 229
    extensions 3
    languages 4
    triggers 2
    virtual views 1
  Server Objects
```



Remember this OLD



Postgres: Physical Architecture

```
root@noodles-staging /var/lib/docker/vo
```

```
du -sh *
```

```
4.0K PG_VERSION
1.6T base
4.0K current_logfiles
2.7M global
125G log
4.0K pg_commit_ts
4.0K pg_dynshmem
8.0K pg_hba.conf
4.0K pg_ident.conf
16K pg_logical
28K pg_multixact
4.0K pg_notify
4.0K pg_replslot
4.0K pg_serial
4.0K pg_snapshots
4.0K pg_stat
4.0K pg_stat_tmp
60K pg_subtrans
4.0K pg_tblspc
4.0K pg_twophase
6.1G pg_wal
18M pg_xact
4.0K postgresql.auto.co
32K postgresql.conf
4.0K postmaster.opts
4.0K postmaster.pid
```

```
root@noodles-staging:/var/
cd base
```

```
root@noodles-staging /var/
```

```
du -sh * | sort -rh
```

```
1.6T 17836
11M 17837
9.0M 5
8.8M 1
7.2M 4
20K pgsq_l_tmp
```

OID of 'internal' db

data folder

system catalogs

log folder
(error, slow
queries,...)

tablespace

wal log

config file

```
root@noodles-staging /var/lib/docker/volumes/deploy_postgres-data/_data
ll
```

```
total 864K
```

```
-rw----- 1 70 70 3 Nov 20 08:35 PG_VERSION
drwx----- 8 70 70 4.0K Nov 20 13:00 base
-rw----- 1 70 70 46 Dec 24 17:40 current_logfiles
drwx----- 2 70 70 4.0K Dec 24 10:09 global
drwx----- 2 70 70 672K Dec 24 17:40 log
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_commit_ts
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_dynshmem
-rw----- 1 70 70 5.7K Nov 20 08:35 pg_hba.conf
-rw----- 1 70 70 2.6K Nov 20 08:35 pg_ident.conf
drwx----- 4 70 70 4.0K Dec 24 18:14 pg_logical
drwx----- 4 70 70 4.0K Nov 20 08:35 pg_multixact
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_notify
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_replslot
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_serial
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_snapshots
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_stat
drwx----- 2 70 70 4.0K Dec 2 15:17 pg_stat_tmp
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_subtrans
drwx----- 2 70 70 4.0K Dec 24 18:13 pg_tblspc
drwx----- 2 70 70 4.0K Nov 20 08:35 pg_twophase
drwx----- 3 70 70 56K Dec 24 18:13 pg_wal
drwx----- 2 70 70 4.0K Dec 24 11:29 pg_xact
-rw----- 1 70 70 88 Nov 20 08:35 postgresql.auto.conf
-rw----- 1 70 70 30K Nov 20 08:35 postgresql.conf
-rw----- 1 70 70 73 Dec 2 15:17 postmaster.opts
-rw----- 1 70 70 94 Dec 2 15:17 postmaster.pid
```



Postgres: Physical Architecture

Timescaledb
chunk data

Data store for large
column (JSON, TEXT,...)

```
SELECT oid, relname, relnamespace, reltuples, reltoastrelid
FROM pg_class
where oid in (100163, 100166, 100167, 100168, 100172);
```

	oid	relname	relnamespace	reltuples	reltoastrelid
1	100163	_hyper_7_799_chunk	17113	7177	100166
2	100166	pg_toast_100163	99	-1	0
3	100167	pg_toast_100163_index	99	0	0
4	100168	_hyper_2_800_chunk	17113	183705	100172
5	100172	pg_toast_100168	99	-1	0

	parent_schema	parent_table	chunk_name
1	dex_indexer	aggregator_swap_history	_hyper_7_799_chunk

```
root@noodles-staging /var/lib/docker/volumes/deploy_postgr
ll 17836 | head -10
```

total 1.6T

```
-rw----- 1 70 70 2.8M Nov 21 04:34 100163
-rw----- 1 70 70 24K Nov 21 04:34 100163_fsm
-rw----- 1 70 70 8.0K Nov 21 04:34 100163_vm
-rw----- 1 70 70 0 Nov 21 04:18 100166
-rw----- 1 70 70 8.0K Nov 21 04:18 100167
-rw----- 1 70 70 46M Nov 21 04:34 100168
-rw----- 1 70 70 32K Nov 21 04:34 100168_fsm
-rw----- 1 70 70 8.0K Nov 21 04:34 100168_vm
-rw----- 1 70 70 0 Nov 21 04:18 100172
```

```
root@noodles-staging /var/lib/docker/volumes/deploy_postgr
ls 17836/ | wc -l
```

22883



Postgres Architecture: Conclusion

Q1: Can we speed up moving a DB between hosts by just copying the **base/** folder?

- **No!** It doesn't contain: system catalogs (*global/*), tablespaces (*pg_tblspc/*),...

Q2: How about just copying a single schema/table by file system?

- Still **no!** A schema is just a *logical way* to organize tables
- Each objects in postgresql have their own OID, this OID is difference between two database clusters
- Conclusion: Copy Postgresql files system is **"ALL OR NOTHING"**

Q4: What is the right way to copy Postgresql

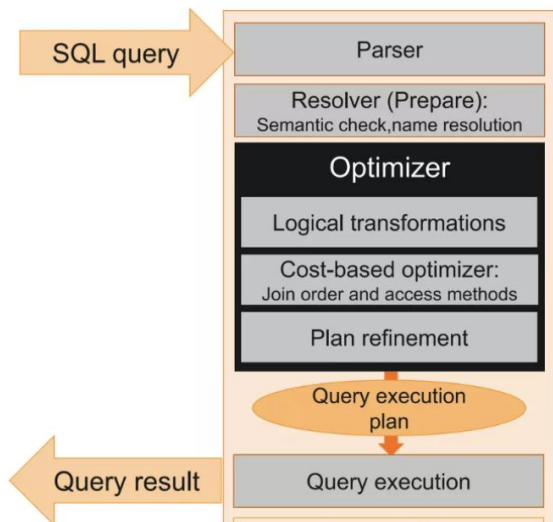
- Small db/schema/table: pg_dump: Logical copy, very slow, generate .sql (INSERT INTO ...)
- Large tables: use `psql -h source_host -d db -c "COPY tableA TO STDOUT (FORMAT BINARY)" | psql -h target_host -d db -c "COPY tableA FROM STDIN (FORMAT BINARY)"` OR export file (binary, csv)
- Production backup: pg_basebackup: safe, no downtime, copy entire database
- Very large database: Copy all physical files, accept downtime.

Q3: What are the takeaways regarding **pg_toast_<oid>**

- Lesson and learn: Use the smallest data types possible to avoid pg_toast, db size, index, group by faster,...



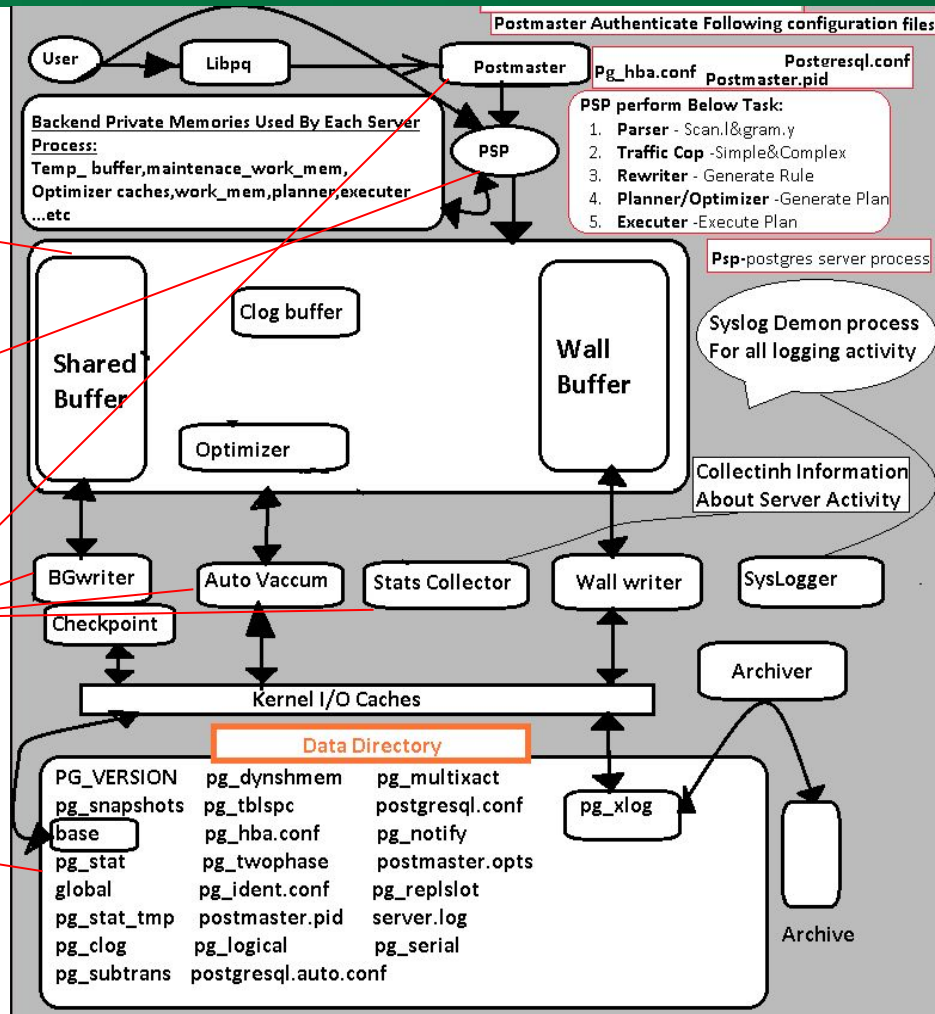
X-Ray Vision



Memory

Process

Disk





Logical Transformations

SELECT * FROM t1, t2 WHERE

t1.a = t2.a AND t2.a = 9 AND (NOT (t1.a > 10 OR t2.b > 3) OR (t1.b = t2.b + 7 AND t2.b = 5));

Negation
elimination

t1.a = t2.a AND t2.a = 9 AND (t1.a <= 10 AND t2.b <= 3 OR (t1.b = t2.b + 7 AND t2.b = 5));

Equality/const
propagation

t1.a = 9 AND t2.a = 9 AND (9 <= 10 AND t2.b <= 3 OR (t1.b = 5 + 7 AND t2.b = 5));

Evaluate const
expressions

t1.a = 9 AND t2.a = 9 AND (9 <= 10 AND t2.b <= 3 OR (t1.b = 12 AND t2.b = 5));

Trivial condition
removal

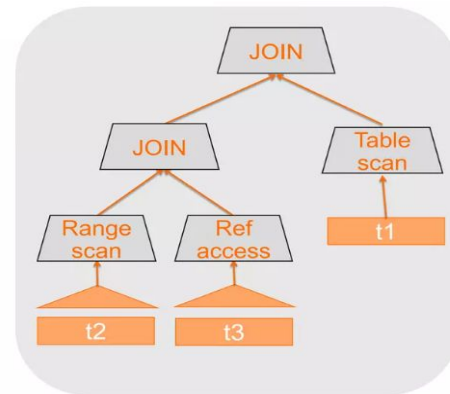
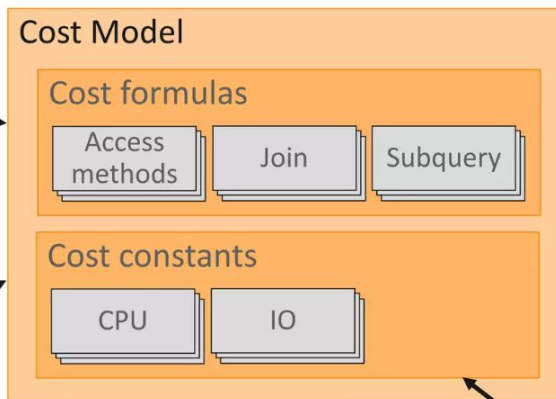
=TRUE

t1.a = 9 AND t2.a = 9 AND (t2.b <= 3 OR (t1.b = 12 AND t2.b = 5));



Cost-based Optimizer

```
SELECT t1.a, t2.z  
FROM t1  
INNER JOIN t2 ON t1.a  
= t2.b  
INNER JOIN t3 ON t3.c  
= t2.d  
WHERE t1.x > 10  
AND t2.y = 100
```

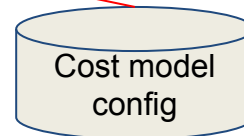


Metadata:

- Record, index size
- Index information
- Uniqueness

Statistics:

- Table size
- Cardinality
- Range estimates
- Histograms





Cost-based Optimizer

'run cost' = 'cpu run cost' + 'disk run cost'

$$= (\text{cpu_tuple_cost} + \text{cpu_operator_cost}) \times N_{\text{tuple}} + \text{seq_page_cost} \times N_{\text{page}}$$

```
explain analyse
```

```
select * from noodles_oltp.latest_coin_metric lcm  
where lcm.liquidity_usd > 100000
```

y > postgres > config > postgresql.conf

- Planner Cost Constants -

```
#seq_page_cost = 1.0           # measured on an arbitrary scale  
#random_page_cost = 4.0       # same scale as above  
#cpu_tuple_cost = 0.01        # same scale as above  
#cpu_index_tuple_cost = 0.005  # same scale as above  
#cpu_operator_cost = 0.0025    # same scale as above  
#parallel_setup_cost = 1000.0  # same scale as above  
#parallel_tuple_cost = 0.1     # same scale as above  
#min_parallel_table_scan_size = 8MB  
#min_parallel_index_scan_size = 512kB
```

QUERY PLAN

```
1 Seq Scan on latest_coin_metric lcm (cost=0.00..24459.14 rows=260 width=138)  
2 Filter: (liquidity_usd > '100000'::double precision)  
3 Rows Removed by Filter: 55963  
4 Planning Time: 0.849 ms  
Execution Time: 80.944 ms
```

$$\text{cost} = 23759 + (0.01 + 0.0025) \times 56011 = 24459.14$$

```
SELECT relpages, reltuples  
FROM pg_class  
WHERE relname = 'latest_coin_metric';
```

Output	internal.pg_catalog.pg_class
relpages	reltuples
23759	56011

QUERY PLAN

```
Seq Scan on latest_coin_metric lcm (cost=0.00..9867.14 rows=259 width=138)  
Filter: (liquidity_usd > '100000'::double precision)  
Rows Removed by Filter: 55963  
Planning Time: 0.464 ms  
Execution Time: 13.851 ms
```

```
vacuum full noodles_oltp.latest_coin_metric;  
SELECT relpages, reltuples  
FROM pg_class  
WHERE relname = 'latest_coin_metric';
```

Output	internal.pg_catalog.pg_class	internal.pg_ca
relpages	reltuples	
1175	56011	

vacuum full

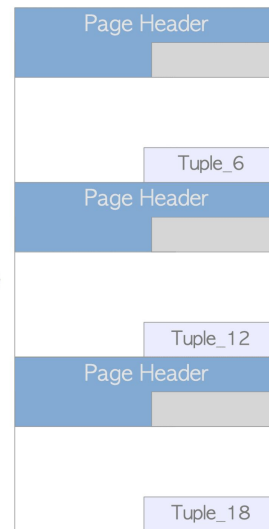
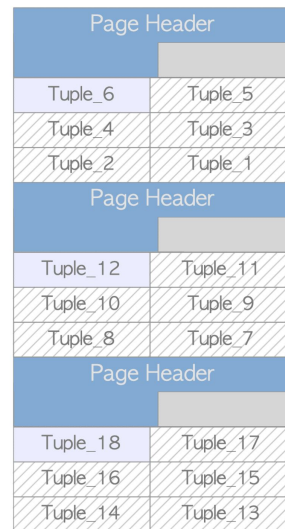


Condition to trigger Autovacuum:

$$n_dead_tuples > autovacuum_vacuum_threshold + autovacuum_vacuum_scale_factor \times reltuples$$

oy > postgres > config > postgresql.conf

```
#-----  
# AUTOVACUUM  
#-----  
  
#autovacuum = on          # Enable autovacuum subprocess? 'on'  
#                          # requires track_counts to also be on.  
#autovacuum_max_workers = 3 # max number of autovacuum subprocesses  
#                          # (change requires restart)  
#autovacuum_naptime = 1min # time between autovacuum runs  
#autovacuum_vacuum_threshold = 50 # min number of row updates before  
#                               # vacuum  
#autovacuum_vacuum_insert_threshold = 1000 # min number of row inserts  
#                               # before vacuum; -1 disables insert  
#                               # vacuums  
#autovacuum_analyze_threshold = 50 # min number of row updates before  
#                               # analyze  
#autovacuum_vacuum_scale_factor = 0.2 # fraction of table size before vacuum  
#autovacuum_vacuum_insert_scale_factor = 0.2 # fraction of inserts over table  
#                               # size before insert vacuum  
#autovacuum_analyze_scale_factor = 0.1 # fraction of table size before analyze
```



 dead tuple



X-Ray Vision: Conclusion

SQL - Structured Query Language

- SQL: a way for devs to describe *what devs want*, not *how the database do*
- A query must go several stages: analysis, transformation, optimization before it is executed. Actual execution order within the db often different with SQL

DBA: Always optimize *postgres.conf* for specific use case

- See: [Minswap Performance W1 slide](#)
- **OOM**: Check Memory allocate, max connection, work mem,... **Full disk**: Delete files, logs,... **Full CPU**: Set up interval job process, config auto vacuum, check slow queries,...

Developers: **EXPLAIN** (ANALYZE, COSTS, VERBOSE, BUFFERS)

- Need to deep understand about Execution plan of a query
- Tool: <https://statics.noodles.fi/pg-dev/pg-explainer-tool.html>

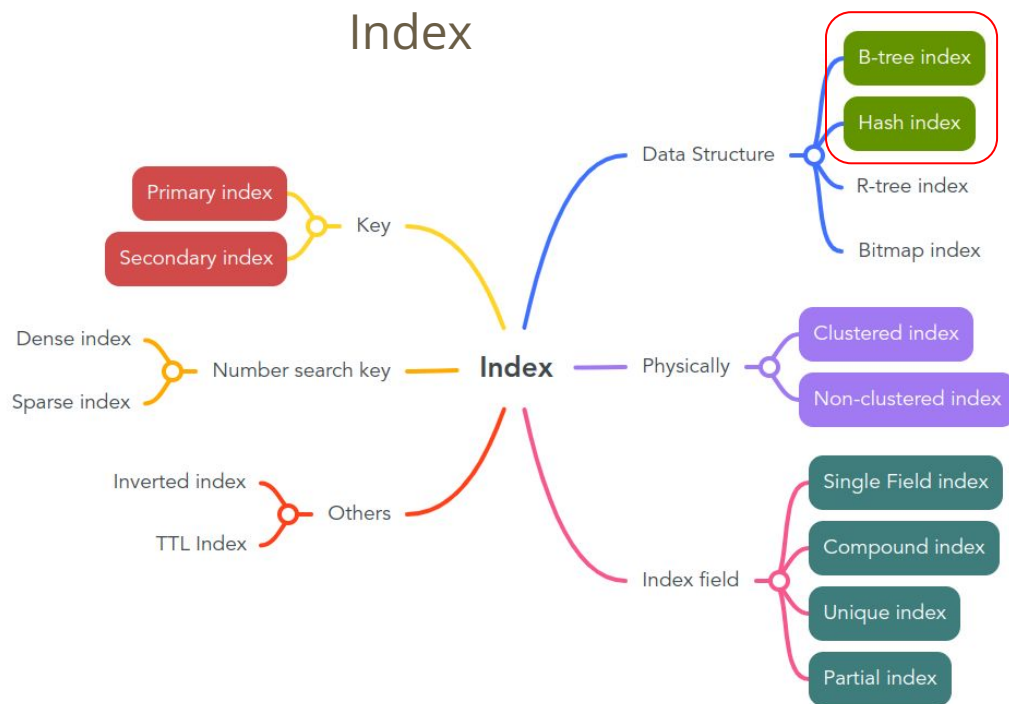
2

Query Optimization





Query Optimization - Basic concept



Partition

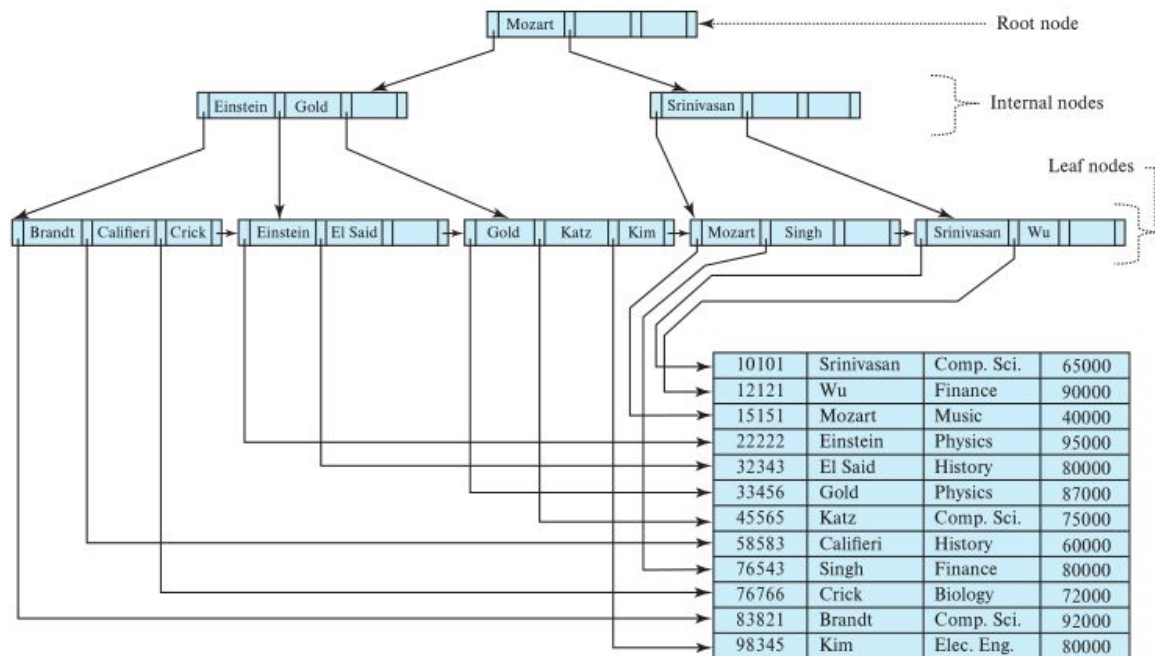
internal	5 of 16
_timescaledb_internal	
tables 2951	
> _hyper_1_2_chunk	based on (checkpoint)
> _hyper_1_11_chunk	based on (checkpoint)
> _hyper_1_18_chunk	based on (checkpoint)
> _hyper_1_24_chunk	based on (checkpoint)
> _hyper_1_30_chunk	based on (checkpoint)
> _hyper_1_35_chunk	based on (checkpoint)
> _hyper_1_40_chunk	based on (checkpoint)
> _hyper_1_46_chunk	based on (checkpoint)
> _hyper_1_51_chunk	based on (checkpoint)
> _hyper_1_56_chunk	based on (checkpoint)
> _hyper_1_61_chunk	based on (checkpoint)



B+-tree index

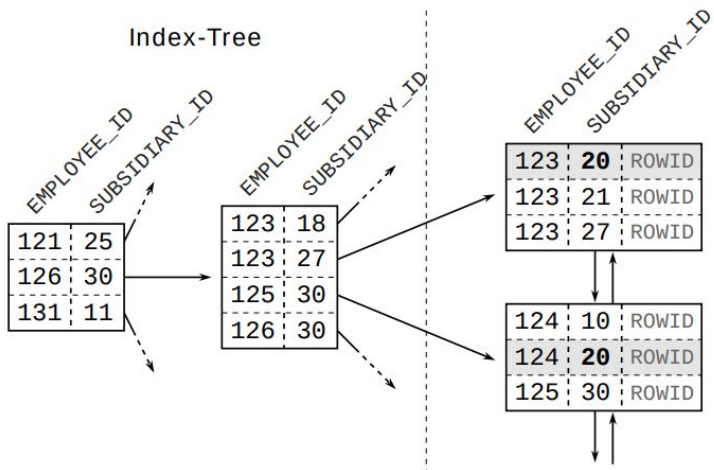
B+-tree data structure

- Every node holds n keys along with pointers to the levels below.
- The leaf nodes store data in a sorted sequence (from smallest to largest)
- Each entry in the nodes includes a pointer to the actual underlying data.
- Leaf nodes form a doubly linked list for faster range scans and bi-directional traversal





Queries match with B-tree index



Match full value

- ✓ WHERE Employee_id = 123
- ✓ WHERE Subsidiary_id = 20 AND Employee_id = 123

Match leftmost prefix

- ✓ WHERE Employee_id = 123
- ✗ WHERE Subsidiary_id = 20

Match range condition

- ✓ WHERE Employee_id >= 123 AND Employee_id < 131
- ✗ WHERE Subsidiary_id > 20

Match exactly in left and range in right columns

- ✓ WHERE Employee_id = 123 AND Subsidiary_id < 20
- ✗ WHERE Employee_id > 123 AND Subsidiary_id = 20



Queries match with B-tree index

LIKE 'WI%ND'	LIKE 'WIN%D'	LIKE 'WINA%'
WIAW	WIAW	WIAW
WIBLQQNPUA	WIBLQQNPUA	WIBLQQNPUA
WIBYHSNZ	WIBYHSNZ	WIBYHSNZ
WIFMDWUQMB	WIFMDWUQMB	WIFMDWUQMB
WIGLZX	WIGLZX	WIGLZX
WIH	WIH	WIH
WIHTFVZNLC	WIHTFVZNLC	WIHTFVZNLC
WIJYAXPP	WIJYAXPP	WIJYAXPP
WINAND	WINAND	WINAND
WINBKYSKW	WINBKYSKW	WINBKYSKW
WIPOJ	WIPOJ	WIPOJ
WISRGPK	WISRGPK	WISRGPK
WITJIVQJ	WITJIVQJ	WITJIVQJ
WIW	WIW	WIW
WIWGPJMQGG	WIWGPJMQGG	WIWGPJMQGG
WIWKHLBJ	WIWKHLBJ	WIWKHLBJ
WIYETHN	WIYETHN	WIYETHN
WIYJ	WIYJ	WIYJ

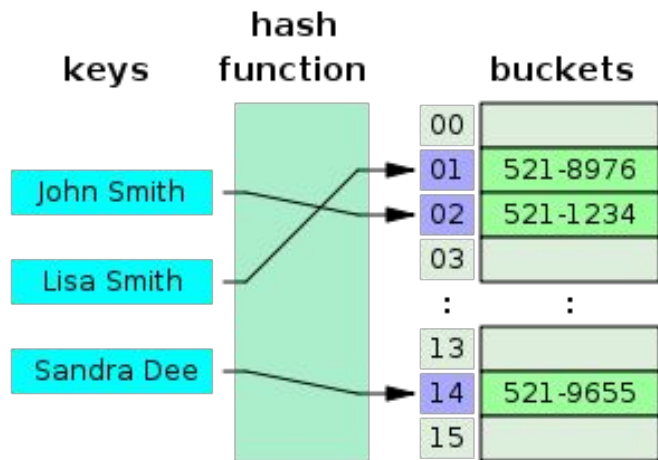
LIKE Operator (Left mode prefix)

✓ WHERE Name like 'WI%ND'

✗ WHERE Name like '%WIND%'



Hash Index



Hash Index: Uses a Hash Table + Hash Function for $O(1)$ lookups



Very fast for **equal comparison** "="



Higher storage overhead than B-trees. (Needs extra space for hash values, bucket arrays, etc.)



Cannot be utilized for **range comparisons**, **sorting**, or **grouping** operations



3

Use Cases



Why your SELECT is slow?

-- auto-generated definition

```
create table coin_trade_history
(  
  id  
  coin_id  
  from_coin_id  
  to_coin_id  
  pool_id  
  sender  
  checkpoint  
  tx_digest  
  created_at  
  primary key (id, created_at)  
);
```

```
id          bigint          not null,  
coin_id     text          not null,  
from_coin_id text        not null,  
to_coin_id  text          not null,  
pool_id     char(66)      not null,  
sender      char(66)      not null,  
checkpoint  bigint        not null,  
tx_digest   char(44)      not null,  
created_at  timestamp with time zone not null,  
primary key (id, created_at)
```

```
create index coin_trade_history_sender_idx  
on coin_trade_history using hash (sender);
```

```
create index coin_trade_history_created_at_idx  
on coin_trade_history (created_at desc);
```

```
create index coin_trade_history_coin_id_created_at_idx  
on coin_trade_history (coin_id asc, created_at desc);
```

```
explain  
select * from dex indexer.coin trade history cth  
where cth.sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'  
and cth.coin_id =  
'0xdba344672e30cb065b1f93e3ab55318768fd6fef66c15942c9f7cb846e2f900e7::usdc::USDC'  
ORDER BY cth.id DESC LIMIT 1
```

QUERY PLAN

Limit (cost=143.92..82886.98 rows=1 width=518)

-> Merge Append (cost=143.92..822383448.60 rows=9939 width=518)

Sort Key: cth_1.id DESC

-> Index Scan Backward using "5_5_coin_trade_history_pkey" on _hyper_4_5_chunk cth_1 (cost=0.29..25942.36 rows=1 width=521)

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "10_10_coin_trade_history_pkey" on _hyper_4_10_chunk cth_2 (cost=0.42..207809.58 rows=1 width=5.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "17_17_coin_trade_history_pkey" on _hyper_4_17_chunk cth_3 (cost=0.42..189893.17 rows=1 width=5.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "23_23_coin_trade_history_pkey" on _hyper_4_23_chunk cth_4 (cost=0.29..133452.90 rows=1 width=5.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "29_29_coin_trade_history_pkey" on _hyper_4_29_chunk cth_5 (cost=0.29..82386.84 rows=1 width=53.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "32_31_coin_trade_history_pkey" on _hyper_4_32_chunk cth_6 (cost=0.29..115584.71 rows=1 width=5.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.

-> Index Scan Backward using "38_38_coin_trade_history_pkey" on _hyper_4_38_chunk cth_7 (cost=0.29..127958.71 rows=1 width=5.

Filter: ((sender = '0xe9008ec0faa85024b357ebb7aa80cfc081ecf5a77bca299e5b365c627f1bb466'::bpchar) AND (coin_id = '0xdba34.



Why your SELECT is slow?

New Plan - Dec 25, 2025, 11:49:23 PM

Plan Grid Raw Query Stats

Execution time: N/A Planning time: N/A

time rows estimation cost buffers IO

#1 Limit

#2 Merge Append

#3 Index Scan Backward

#4 Index Scan Backward

#5 Index Scan Backward

#6 Index Scan Backward

#7 Index Scan Backward

#8 Index Scan Backward

#9 Index Scan Backward

#10 Index Scan Backward

#11 Index Scan Backward

#12 Index Scan Backward

#13 Index Scan Backward

#14 Index Scan Backward

#15 Index Scan Backward

#16 Index Scan Backward

#17 Index Scan Backward

#18 Index Scan Backward

#19 Index Scan Backward

#20 Index Scan Backward

#21 Index Scan Backward

#22 Index Scan Backward

#23 Index Scan Backward

#24 Index Scan Backward

#25 Index Scan Backward

#26 Index Scan Backward

#27 on _hyper_4_681_chunk as cth_124

#28 using ""681_680_coin_trade_history_

#29 Index Scan Backward

#30 Index Scan Backward

#31 Index Scan Backward

#32 Index Scan Backward

#33 Index Scan Backward

#34 Index Scan Backward

#35 Index Scan Backward

#36 Index Scan Backward

#37 Index Scan Backward

#38 Index Scan Backward

#39 Index Scan Backward

#40 Index Scan Backward

#41 Index Scan Backward

#42 Index Scan Backward

#43 Index Scan Backward

#44 Index Scan Backward

#45 Index Scan Backward

#46 Index Scan Backward

#47 Index Scan Backward

#48 Index Scan Backward

Limit Node returns a specified number of rows from a record set.

General IO & Buffers Output Workers Misc

Timing: -

Rows: N/A (Planned: 1)

Cost: 0.00 (Total: 82,880.60)

Merge Append by cth_1.id DESC

General IO & Buffers Output Workers Misc

Timing: -

Rows: N/A (Planned: 9,940)

Cost: 401,007,014.75 (Total: 822,402,745.54)

Index Scan Backward #126 on _hyper_4_686_chunk as cth_125 using ""686_685_coin_trade_history_pkey""

Index Scan Node finds relevant records based on an Index. Index Scans perform 2 read operations: one to read the index and another to read the actual value from the table.

General IO & Buffers Output Workers Misc

Timing: -

Rows: N/A (Planned: 1)

Cost: 332,898.00 (Total: 332,898.00)

Index Scan Backward #128 on _hyper_4_691_chunk as cth_126 using ""691_691_coin_trade_history_

Index Scan Backward #129 on _hyper_4_695_chunk as cth_127 using ""695_695_coin_trade_history_

Index Scan Backward #130 on _hyper_4_701_chunk as cth_128 using ""701_701_coin_trade_history_

Index Scan Backward #131 on _hyper_4_706_chunk as cth_129 using ""706_706_coin_trade_history_

Merge Append huge cost

Why use index coin_trade_history_pkey?



Why your SELECT is slow?

```
explain analyse
select * from dex indexer.coin_trade_history cth
where cth.coin id =
'0x087d928a6166747c0eeb02abd7c6f8e75675340afbabb0847047c670fbc7be4f::james::JAMES'
and cth.created at between '2024-12-14T00:00:00+00' and '2025-12-14T00:00:00+00'
ORDER BY cth.id DESC LIMIT 20;
```

Still use coin_trade_history_pkey?

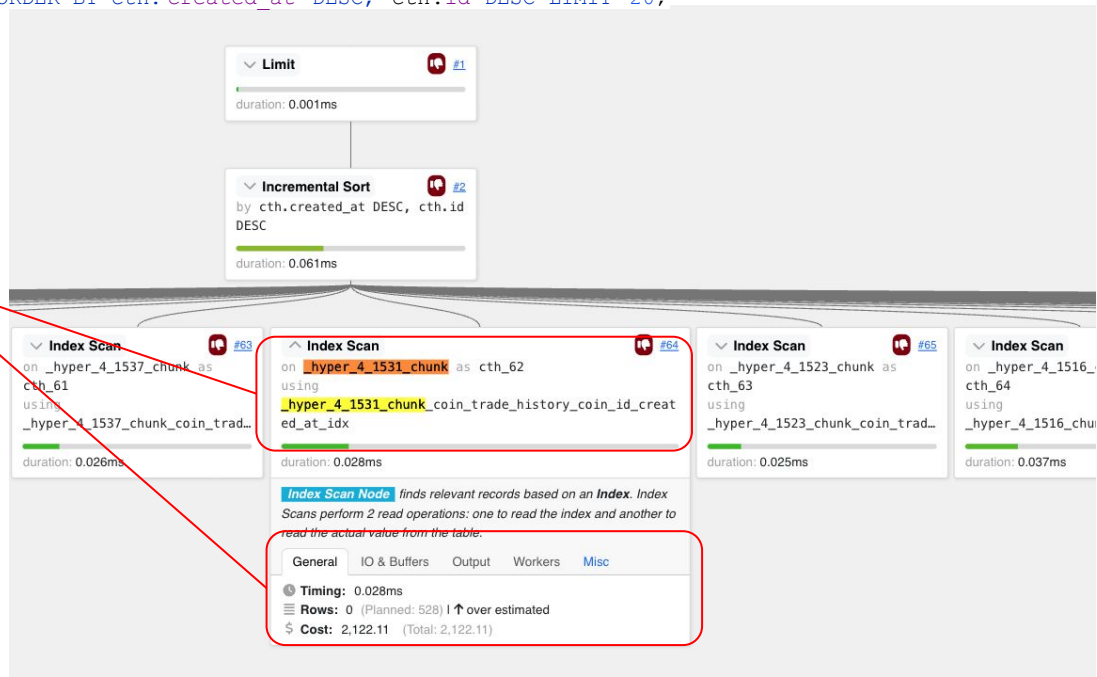




Why your SELECT is slow?

```
explain analyse
select * from dex indexer.coin_trade_history cth
where cth.coin_id =
'0x087d928a6166747c0eeb02abd7c6f8e75675340afbabb0847047c670fbc7be4f::james::JAMES'
and cth.created_at between '2024-12-14T00:00:00+00' and '2025-12-14T00:00:00+00'
ORDER BY cth.created_at DESC, cth.id DESC LIMIT 20;
```

coin_id_created_at_idx





Why your SELECT is slow? Conclusion

Q1: Best practice for partition/hypertable base table?

- Always try to put partition key into our query. If your queries don't use partition key (created_at) at most cases -> Should not use hypertable

Q2: Best practice for indexes in a table?

- Prefer multi-column index instead of multi single indexes
- What is different between *idx1(created_at, coin_id)* and *idx2(coin_id, created_at)*
- Give Postgres Optimizer a few hints so it doesn't pick a bad plan

Use EXPLAIN to dive deep into what Postgres is actually doing under the hood

- [Ads] <https://statics.noodles.fi/pg-dev/pg-explainer-tool.html>



Why your INSERT is slow?



Patrik Dec 12th at 8:47 PM

@Richard Nguyen @Link @Long @Thanh Hai

Given the overload of noodles-prod-3 alerts regarding disk usage I checked what's up.

PostgreSQL `internal` database has written **75TB of temp files** since Nov 20 due to queries exceeding `work_mem` (currently 16MB). With 5 parallel workers per query it causes operations spill to disk constantly.

I think if we increase the `work_mem` to 128-256MB, it will keep more operations in memory and not stress out the disk so much, not to mention it should also make the queries faster.

```
for _, coin := range updateCoins {
    if err := tx.
        Where("coin_ident = ?", coin.CoinIdent.Id()).
        Updates(&coin).Error; err != nil {
        tx.Rollback()
        return err
    }
}
```

Prefer bulk insert instead of
single-row insert in for loop



10x faster

```
if err := tx.Clauses(clause.OnConflict{
    Columns:   clauseColumns,
    DoUpdates: clause.AssignmentColumns(gormutil.GetNonPrimaryColumns(coin)),
}).CreateInBatches(data, 500).Error; err != nil {
    tx.Rollback()
    return err
}
```



Why your INSERT is slow? <Bonus>

```
SELECT coin_id,
  SUM(CASE WHEN action = 'buy' THEN amount_in_usd ELSE 0 END) AS buy_volume,
  SUM(CASE WHEN action = 'sell' THEN amount_in_usd ELSE 0 END) AS sell_volume,
  COUNT(DISTINCT CASE WHEN action = 'buy' THEN tx_digest END) AS tx_buy,
  COUNT(DISTINCT CASE WHEN action = 'sell' THEN tx_digest END) AS tx_sell,
  COUNT(DISTINCT sender) FILTER (WHERE action = 'buy') AS unique_buyers,
  COUNT(DISTINCT sender) FILTER (WHERE action = 'sell') AS unique_sellers,
  COUNT(DISTINCT sender) AS unique_traders,
  COUNT(DISTINCT tx_digest) AS unique_tx
FROM dex_indexer.coin_trade_history
WHERE coin_id IN (%s)
AND created_at >= ? AND created_at <= ?
GROUP BY coin_id;
```

Select also have disk I/O write!



Link Dec 15th at 11:26 AM

@Patrik, this issue has been resolved. TLDR:

The main issues were slow queries on the data worker job (both read and write) caused by:

- Heavy Write Operations: In `noodles_oltp.coin`, `noodles_oltp.latest_coin_metric`, and `noodles_oltp.latest_pool_metric`.
- Heavy Read Operations: In `dex_indexer.coin_trade_history`. (This was a read operation, but it also had IO writes due to sorting/grouping op).

Why your solution was not working:

- `work_mem`: 16MB is already large enough for an OLTP database. Increasing this might cause OOM issue if we cannot control connections from our BE web application. Slow query in `coin_trade_history` actually created 1GB of data per session, and flushed to disk for sorting/grouping (this occurs because the large dataset must aggregate distinct wallets, total buy/sell by wallet, and `coin_id` within a 7-day period) -> This data should be cached
- `dex_indexer.pair_trade_history (pool_id, action, created_at DESC, id DESC)` This index includes too many columns and is not our bottleneck

Best Practices that I was implemented:

- Always prefer batch upsert over single record update/insert within a `for` loop. (`noodles_oltp.latest_coin_metric`, `noodles_oltp.latest_pool_metric`)
- Cache heavy data that is not frequently updated or spans a long period (`dex_indexer.coin_trade_history`)
- Prefer preprocessing data in the application layer, and then only flush changes/updates to the database (`noodles_oltp.coin`)
- Bonus: Recreate the index `coin_trade_history(coin_id, created_at)` instead of `coin_trade_history(coin_id)` for better index scan



Why your INSERT is slow?

Noodles Indexer write problems

```
private async executeBulkInsert(
  tableName: string,
  args: Record<string, any>[],
  onConflictDoNothing?: boolean,
): Promise<void> {
  if (args.length === 0) {
    return;
  }
  const columns: Record<string, any>[] = [];
  for (const arg of args) {
    const record: Record<string, any> = {};
    for (const [key, value] of Object.entries(arg)) {
      record[key] = value;
    }
    columns.push(record);
  }
  const rawSql = buildRawInsert(tableName, columns, onConflictDoNothing);
  if (Maybe.isNothing(rawSql)) {
    return;
  }
  await this.prisma.$executeRawUnsafe(rawSql);
}
```

Build raw insert
(Don't do this at home!)

parallel insert
outside transaction

```
async insertDexData(indexingData: DexIndexingData[]): Promise<void> {
  newMetastablePoolCreationInputs,
  metastableLiquidityHistoryInputs,
} = this.prepareInsertDexData(indexingData);

const tasks: Promise<unknown>[] = [];

// Only create promises for non-empty arrays to reduce overhead
if (decimalHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.coin_decimal_history", decimalHistoryInputs));
}
if (coinMetricHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.coin_metric_history", coinMetricHistoryInputs));
}
if (poolMetricHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.pool_metric_history", poolMetricHistoryInputs));
}
if (poolCreationInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.pool_creation", poolCreationInputs, true));
}
if (pairTradeHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.pair_trade_history", pairTradeHistoryInputs));
}
if (coinTradeHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.coin_trade_history", coinTradeHistoryInputs));
}
if (aggregatorSwapHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.aggregator_swap_history", aggregatorSwapHistoryInputs));
}
if (newSteammCoinInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.steamm_coin_mapping", newSteammCoinInputs, true));
}
if (newSuiStakedCoinInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.staked_coin", newSuiStakedCoinInputs, true));
}
if (newMetastablePoolCreationInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.metastable_pool_creation", newMetastablePoolCreationInputs, true));
}
if (metastableLiquidityHistoryInputs.length > 0) {
  tasks.push(this.executeBulkInsert("dex_indexer.metastable_liquidity_history", metastableLiquidityHistoryInputs));
}

await Promise.all(tasks);
}
```



Why your INSERT is slow? Conclusion

What actually slows INSERT?

- WAL generation & fsync -> postgres.conf optimize (wal size, synchronous_commit, shared_buffers, checkpoint_timeout...)
- Index maintenance -> reduce index size, data type
- Lock acquisition -> Reduce overhead write in multi process, Avoid concurrent DDL
- MVCC metadata
- Constraint & trigger checks -> reduce constraints, triggers
- Network
- ORM, Parsing overhead -> raw queries
- Write to disk by pages -> bulk insert

Others tips

- COPY: the fastest way to insert (`COPY coin_indexer.coin FROM STDOUT (FORMAT BINARY)`)
- WAL & UNLOGGED tables (`ALTER TABLE noodles_oltp.latest_coin_metric SET UNLOGGED`)
- Isolation level: (`READ UNCOMMITTED`, `READ COMMITTED`, `REPEATABLE READ`, `SERIALIZABLE`)

=> Fast INSERT = less work per row



Why Postgres UPDATE/DELETE is expensive

Key idea: PostgreSQL never updates rows in place!

- UPDATE = **INSERT** new row + mark old row dead
- DELETE = mark row dead
- Old versions remain until VACUUM

=> This is the cost of **MVCC**

Why UPDATE gets slower over time

- Table grows, but data doesn't
- Dead tuples, indexes point to dead rows, more heap & index scans

=> This is called **BLOAT**

VACUUM

- VACUUM (normal): Removes dead tuples logically, marks space reusable
- VACUUM FULL (dangerous but powerful): Rewrites entire table, physically shrinks files, Requires ACCESS EXCLUSIVE lock
- VACUUM ANALYZE: Collects statistics, improves query plans

=> Let autovacuum handle most cases, manual vacuum hot tables



Why Postgres UPDATE/DELETE is expensive

Key idea: PostgreSQL never updates rows in place!

- UPDATE = **INSERT** new row + mark old row dead
- DELETE = mark row dead
- Old versions remain until VACUUM

=> This is the cost of **MVCC**

Why UPDATE gets slower over time

- Table grows, but data doesn't
- Dead tuples, indexes point to dead rows, more heap & index scans

=> This is called **BLOAT**

VACUUM

- VACUUM (normal): Removes dead tuples logically, marks space reusable
- VACUUM FULL (dangerous but powerful): Rewrites entire table, physically shrinks files, Requires ACCESS EXCLUSIVE lock
- VACUUM ANALYZE: Collects statistics, improves query plans

=> Let autovacuum handle most cases, manual vacuum hot tables



UPDATE/DELETE - Conclusion

Best practices

- Same with INSERT: Always consider bulk UPDATE/DELETE
- Long-running transactions: silent killer -> reduce long running transactions to reduce lock/deadlock
- DELETE vs partition drop -> DELETE is expensive,
 - Bad: `DELETE FROM xxx WHERE created_at <'2025-12-01' AND created_at >'2025-11-01';`
 - Good: `DROP TABLE _hyper_3_1208_chunk;`
- UNLOGGED tables (Disabled in Postgres 20): `CREATE UNLOGGED TABLE t (...);`
- Tune Autovacuum

Others tips

- Update-heavy -> redesign
- Time-series -> partition
- Bulk changes -> rewrite

Thank
you





References

- PostgreSQL official docs: <https://www.postgresql.org/docs/> (Highly recommended)
- The Internals of PostgreSQL: <https://www.interdb.jp/pg/pgsql01/index.html> (Deep and dive)
- [noodles.fi](#) source code & data